

数据库系统实验报告

学号： 23009201247 姓名： 郑墨涵

手机： 邮箱：

阿里云账号：

完成时间： 2025 年 12 月 28 日

实验 1 数据库远程连接

实验目的 熟悉 PolarDB for MySQL 使用方式，了解阿里云 dms 实验平台，并学会使用 Python 或 Java 语言实现对 数据库的远程连接操作。

实验步骤

- 1. 登录阿里云 dms 控制台，dms 控制台网址为 <https://dmslab.aliyun.com/>
- 2. 创建阿里云账号
- 3. 申请使用 PolarDB for MySQL 并创建集群：

总数量

1

运行中

1

即将到期

0

已过期

0

立即购买

续费

无感数据集成

标签筛选

不分组

支持ID/名称模糊搜索

☐	ID/名称	运行状态	标签	所属产品	地域	付费类型	创建时间	操作
☐	pc-bp1bg1q2o2ma353b6 pc-bp1bg1q2o2ma353b6	运行中		PolarDB MySQL	华东1（杭州）	按量付费	2025年12月28日 17:	详情 登录 转包年包月

☐

设置标签

☐

解绑标签

每页显示

5

共有1条

< 上一页

1

下一页 >

- 4. 登录数据库，创建数据库账号：

← pc-bp1bg1q2o2ma353b6 运行中		登录数据库	AI 实例巡检	数据传输	更多
用户账号					
创建账号 自定义权限 模糊搜索（当前页） 请输入					
账号名	状态	所属数据库	备注	类型	操作
znh	可用		- 2	高权限账号	重置权限 修改密
每页显示: 30 条					

- 5. 创建表：

```

CREATE TABLE
student (
  sno VARCHAR(10) PRIMARY KEY,
  sname VARCHAR(10) NOT NULL,
  ssex CHAR(1) CHECK (ssex IN ('M', 'F')),
  sage INTEGER CHECK (sage > 0),
  sdept VARCHAR(20)
);

CREATE TABLE
course (
  cno VARCHAR(10) PRIMARY KEY,
  cname VARCHAR(20) NOT NULL,
  cpno VARCHAR(10),
  ccredit INTEGER,
  FOREIGN KEY (cpno) REFERENCES course (cno)
);

CREATE TABLE
sc (
  sno VARCHAR(10),
  cno VARCHAR(10),
  grade INTEGER,
  PRIMARY KEY (sno, cno),
  FOREIGN KEY (sno) REFERENCES student (sno),
  FOREIGN KEY (cno) REFERENCES course (cno)
);

```

6. 在表中插入数据

```

INSERT INTO
student (sno, sname, ssex, sage, sdept)
VALUES
('2021001', '张三', 'M', 20, '计算机系'),
('2021002', '李四', 'F', 19, '计算机系');

```

7. 开启安全访问代理

当前实例: pc-bp1bg1q2o2ma353b6.mysql.polardb.rds.aliyuncs.com:3306 【pc-bp1bg1q2o2ma353b6】

关闭安全访问代理

关闭后，您的数据库实例将失去JDBC协议方式的安全代理能力，所有的授权信息都会被回收。
 [查看帮助文档](#)

基本信息

开通人 郑墨涵
 数据库账号 zmh

内网地址 ⑦

MySQL 协议: dphzpl-e
 roxy dms.aliyuncs.com:3306
 示例 复制示例

 HTTP(S) 协议: https://
 6ndrdf.proxy.dms.aliyuncs.com:443/server
 示例 复制示例

公网地址 ⑦ 关闭

MySQL 协议: dphzpl-e
 ub proxy.dms.aliyuncs.com:3306
 示例 复制示例

 HTTP(S) 协议: https://
 6ndrdf-pub.proxy.dms.aliyuncs.com:443/server
 示例 复制示例

授权信息

1. 认证信息是登录安全访问代理的凭证，请妥善保管，不要泄露！
 2. 安全访问代理复用了数据管理DMS内已有的安全规则、数据权限、敏感列配置等，为企业提供更全面的安全管控、访问控制、数据脱敏以及操作审计等数据库安全管理能力。

授权

被授权人	认证信息	授权时间	独立账号 ⑦	授权来源
郑墨涵 (1482066855427126)	AccessID: Dv7P4Uj- AccessSecret: 3NX- Dd9xwedQu1 复制 llllQ4EmaqpV8cLBUah 复制 隐藏	2025-12-28 18:32:26	否	责任人授权 (1482066855427126)

8. 在 connect.py 文件中写入代码

```
import pymysql

connection = pymysql.connect(host='dphzpl-euiymkvuma6ndrdf-pub.proxy.dms.aliyuncs.com',
                             port=3306,
                             user='Dv7P4U54F3iC8uDd9xwedQv1',
                             passwd='3NXE3Pd2Zj4lIIQ4EmagpV8cLBUah',
                             db='zmh')

try:
    with connection.cursor() as cursor:
        sql = "SELECT * FROM `student`"
        cursor.execute(sql)
        for result in cursor:
            print(result)
finally:
    connection.close()
```

9. 运行 python 文件，成功连接。

```
D:\PythonProject\connect\.venv\Scripts\python.exe D:\PythonProject\connect\connect.py
('2021001', '张三', 'M', 20, '计算机系')
('2021002', '李四', 'F', 19, '计算机系')

进程已结束，退出代码为 0
```

实验 2 XDSQL 与 PolarDB 对比实验

实验目的 XDSQL 数据库是自主开发的一款微型数据库系统，具有简单的建库、建表、增删改查等功能，其 SQL 解释层采用 lex 和 yacc 实现，执行层采用 C++实现，存储引擎则是使用最基础的方式实现，即每个数据库对应操作系统的一个文件夹，每个数据表对应操作系统中的一个文件。通过对 XDSQL 数据库和 PolarDB 数据库的测试对比，学生可以了解这两款数据库所支持的 SQL 语句类型，体会在数据库软件中不同 SQL 语句实现的困难度，同时去发现 XDSQL 中存在的问题。

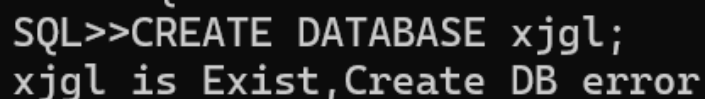
实验步骤 1. 登录阿里云 dms 控制台，进入 PolarDB for MySQL 数据库；
2. 下载 XDSQL 压缩包到本地电脑； 3. 解压压缩包，运行 XDSQL.exe，启动 XDSQL； 4. 在两个数据库内执行测试集内的 SQL 语句，根据运行结果去发现 XDSQL 的不足和不同 SQL 语句实现的难易程度。测试集来自教材内容，具体见下文所示。

SQL 语句测试集

创建数据库

SQL 语句如下：

```
CREATE DATABASE xjgl;
```



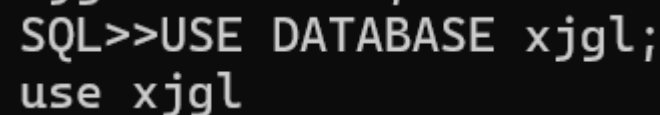
```
SQL>>CREATE DATABASE xjgl;  
xjgl is Exist,Create DB error
```

polarDB 运行失败（普通用户没有权限创建数据库）。XDSQL 运行成功。

切换数据库

SQL 语句如下：

```
USE DATABASE xjgl;
```



```
SQL>>USE DATABASE xjgl;  
use xjgl
```

polarDB 运行失败（普通用户没有权限切换数据库），XDSQL 运行成功。

定义模式/删除模式

polarDB 与 XDSQL 都不支持此类 SQL 语句。

基本表的定义、删除与修改

定义基本表

创建学生表 Student、课程表 Course、学生选课表 SC，SQL 语句如下

```
CREATE TABLE Student(Sno CHAR(9),Sname CHAR(20),Ssex CHAR(2),Sage INT,Sdept  
CHAR(20));
```

```
CREATE TABLE Course(Cno CHAR(4),Cname CHAR(40),Cpno CHAR(4),Ccredit INT);
```

```
CREATE TABLE SC(Sno CHAR(9),Cno CHAR(4),Grade INT);
```

```
SQL>>CREATE TABLE Student(Sno CHAR(9),Sname CHAR(20),Ssex CHAR(2),Sage INT,Sdept CHAR(20));  
create table ok  
SQL>>CREATE TABLE Course(Cno CHAR(4),Cname CHAR(40),Cpno CHAR(4),Ccredit INT);  
create table ok  
SQL>>CREATE TABLE SC(Sno CHAR(9),Cno CHAR(4),Grade INT);  
create table ok
```

删除基本表

删除 Student 表

```
DROP TABLE Student;
```

```
SQL>>DROP TABLE Student;  
drop table ok
```

polarDB 运行成功。XDSQL 运行成功。

修改基本表

向 Student 表增加”出生日期“列，数据类型为日期型

```
ALTER TABLE Student ADD birthdate DATE;
```

```
SQL>>ALTER TABLE Student ADD birthdate DATE;  
语法错误 /不支持本条SQL语句  
语法错误 /不支持本条SQL语句  
语法错误 /不支持本条SQL语句  
语法错误 /不支持本条SQL语句  
语法错误 /不支持本条SQL语句  
语法错误 /不支持本条SQL语句  
语法错误 /不支持本条SQL语句
```

向 Student 表增加 Sno 列为主码的约束条件

```
ALTER TABLE Student ADD PRIMARY KEY(Sno);
```

```
SQL>>ALTER TABLE Student ADD PRIMARY KEY(Sno);
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
```

向 *Student* 表增加 *Sname* 列为唯一值的约束条件

```
ALTER TABLE Student ADD UNIQUE(Sname);
```

```
SQL>>ALTER TABLE Student ADD UNIQUE(Sname);
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
```

以上 SQL 语句，在 polarDB 运行成功，XDSQL 显示“语法错误/不支持本条 SQL 语句”。

完整性约束条件 重新创建带有完整性约束条件的用户数据表。

```
CREATE TABLE Student(Sno CHAR(9) PRIMARY KEY,Sname CHAR(20) UNIQUE,Ssex
CHAR(2),Sage SMALLINT,Sdept CHAR(20));
```

```
CREATE TABLE Course(Cno CHAR(4) PRIMARY KEY,Cname CHAR(40) NOT NULL,Cpno
CHAR(4),Ccredit INT,FOREIGN KEY(Cpno) REFERENCES Course(Cno));
```

```
CREATE TABLE SC(Sno CHAR(9),Cno CHAR(4),Grade INT,PRIMARY KEY(Sno,Cno),FOREIGN
KEY(Sno)REFERENCES Student(Sno),FOREIGN KEY(Cno)REFERENCES Course(Cno));
```


数据查询

以下语句在未加约束的三个表上执行。

插入数据

运行以下 SQL 语句

```
INSERT INTO Student VALUES('201215121','李勇','男',20,'CS');
INSERT INTO Student VALUES('201215122','刘晨','女',19,'CS');
INSERT INTO Student VALUES('201215123','王敏','女',18,'MA');
INSERT INTO Student VALUES('201215124','张立','男',19,'IS');
INSERT INTO COURSE VALUES('1','数据库','5',4);
INSERT INTO COURSE VALUES('3','信息系统','1',4);
INSERT INTO COURSE VALUES('4','操作系统','6',3);
INSERT INTO COURSE VALUES('5','数据结构','7',4);
INSERT INTO COURSE VALUES('7','PASCAL 语言','6',4);
INSERT INTO SC VALUES('201215121','1',92);
INSERT INTO SC VALUES('201215121','2',85);
INSERT INTO SC VALUES('201215121','3',88);
INSERT INTO SC VALUES('201215122','2',90);
INSERT INTO SC VALUES('201215122','3',80);
```

```

SQL>>INSERT INTO Student VALUES('201215121','李勇','男',20,'CS');
insert ok
SQL>>INSERT INTO Student VALUES('201215122','刘晨','女',19,'CS');
insert ok
SQL>>INSERT INTO Student VALUES('201215123','王敏','女',18,'MA');
insert ok
SQL>>INSERT INTO Student VALUES('201215124','张立','男',19,'IS');
insert ok
SQL>>INSERT INTO COURSE VALUES('1','数据库','5',4);
insert ok
SQL>>INSERT INTO COURSE VALUES('3','信息系统','1',4);
insert ok
SQL>>INSERT INTO COURSE VALUES('4','操作系统','6',3);
insert ok
SQL>>INSERT INTO COURSE VALUES('5','数据结构','7',4);
insert ok
SQL>>INSERT INTO COURSE VALUES('7','PASCAL语言','6',4);
insert ok
SQL>>INSERT INTO SC VALUES('201215121','1',92);
insert ok
SQL>>INSERT INTO SC VALUES('201215121','2',85);
insert ok
SQL>>INSERT INTO SC VALUES('201215121','3',88);
insert ok
SQL>>INSERT INTO SC VALUES('201215122','2',90);
insert ok
SQL>>INSERT INTO SC VALUES('201215122','3',80);
insert ok

```

polarDB 运行成功。XDSQL 运行成功。

运行以下语句

```
INSERT INTO COURSE VALUES('2','数学',NULL,2);
```

```
INSERT INTO COURSE VALUES('6','数据处理',NULL,2);
```

```

SQL>>INSERT INTO COURSE VALUES('2','数学',NULL,2);
语法错误/不支持本条 SQL语句
语法错误/不支持本条 SQL语句
语法错误/不支持本条 SQL语句
语法错误/不支持本条 SQL语句
语法错误/不支持本条 SQL语句

SQL>>INSERT INTO COURSE VALUES('6','数据处理',NULL,2);
语法错误/不支持本条 SQL语句
语法错误/不支持本条 SQL语句
语法错误/不支持本条 SQL语句
语法错误/不支持本条 SQL语句
语法错误/不支持本条 SQL语句

```

polarDB 运行成功，XDSQL 显示”语法错误/不支持本条 SQL 语句“。

更新数据

```
UPDATE Student SET Sage=Sage+1;
```

```
SQL>>UPDATE Student SET Sage=Sage+1;  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句
```

运行以下语句 polarDB 运行成功, XDSQL 显示”语法错误/不支持本条 SQL 语句“。

运行以下语句

```
UPDATE Course SET Cpno='1' WHERE Cno='6';
```

```
SQL>>UPDATE Course SET Cpno='1' WHERE Cno='6';  
update 0 rows
```

polarDB 运行成功。XDSQL 运行成功。

删除数据

运行以下 SQL 语句：

```
DELETE FROM SC WHERE Sno='201215121' AND Cno='1';
```

```
DELETE FROM SC;
```

```
SQL>>UPDATE Course SET Cpno='1' WHERE Cno='6';  
update 0 rows  
SQL>>DELETE FROM SC WHERE Sno='201215121' AND Cno='1';  
delete 1 rows  
SQL>>DELETE FROM SC;  
delete 4 rows
```

polarDB 运行成功。XDSQL 运行成功。

查询数据

单表查询

运行以下 SQL 语句：

```
SELECT * FROM Student;
```

```
SELECT * FROM Sc; SELECT * FROM Course;
```

```
SELECT Sno,Sname,Sage FROM Student;
```

```
SELECT Sname FROM Student WHERE Sdept='cs';
```

```
SELECT Sname,Sage FROM Student WHERE Sage<20;
```

```

SQL>>SELECT * FROM Student;
sno      sname      ssex      sage      sdept
201215121 李勇      男        20        CS
201215122 刘晨      女        19        CS
201215123 王敏      女        18        MA
201215124 张立      男        19        IS
Total row: 4
SQL>>SELECT * FROM Sc;
sno      cno      grade
Total row: 0
SQL>>SELECT * FROM Course;
cno      cname      cpno      ccreit
1        数据库    5         4
3        信息系统      1         4
4        操作系统    6         3
5        数据结构    7         4
7        PASCAL语言  6         4
Total row: 5
SQL>>SELECT Sno,Sname,Sage FROM Student;
sno      sname      sage
201215121 李勇      20
201215122 刘晨      19
201215123 王敏      18
201215124 张立      19
Total row: 4
SQL>>SELECT Sname FROM Student WHERE Sdept='cs';
sname
Total row: 0
SQL>>SELECT Sname,Sage FROM Student WHERE Sage<20;
sname      sage
刘晨      19
王敏      18
张立      19
Total row: 3

```

polarDB 运行成功。XDSQL 运行成功。

运行以下 SQL 语句：

```
SELECT Sname,2021-Sage FROM Student;
```

```

SQL>>SELECT Sname,2021-Sage FROM Student;
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句

```

polarDB 运行成功。XDSQL 显示”语法错误/不支持本条 SQL 语句“。

运行以下 SQL 语句

```
SELECT Sname,'Year of birth:',2014-Sage FROM Student;
```

```
SQL>>SELECT Sname, 'Year of birth:',2014-Sage FROM Student;
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
```

polarDB 运行成功。XDSQL 显示”语法错误/不支持本条 SQL 语句“。

运行以下 SQL 语句：

```
SELECT DISTINCT Sno FROM SC;
```

```
SELECT ALL Sno FROM SC;
```

```
SQL>>SELECT DISTINCT Sno FROM SC;
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句

SQL>>SELECT ALL Sno FROM SC;
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
```

polarDB 运行成功。XDSQL 显示”语法错误/不支持本条 SQL 语句”

运行以下 SQL 语句

```
SELECT* FROM Student WHERE Sage BETWEEN 20 AND 21;
```

```
SELECT * FROM Student WHERE Sdept IN('CS','IS');
```

```
SELECT * FROM Student WHERE Sname LIKE '刘%';
```

```
SELECT * FROM Student WHERE Sname LIKE '刘_';
```

```
SQL>>SELECT* FROM Student WHERE Sage BETWEEN 20 AND 21;
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
语法错误/不支持本条SQL语句
```

polarDB 运行成功。XDSQL 显示”语法错误/不支持本条 SQL 语句”

运行以下 SQL 语句

```
SELECT* FROM Student ORDER BY Sdept,Sage DESC;
```

```
SELECT COUNT(*) FROM Student;
```

```
SELECT COUNT(DISTINCT Sno) FROM SC;
```

```
SELECT AVG(Sage) FROM Student;
```

```
SELECT SUM(Ccredit) FROM Course;
```

```
SELECT Sno FROM SC GROUP BY SNO HAVING COUNT(*)>=3;
```

```
SQL>>SELECT* FROM Student ORDER BY Sdept,Sage DESC;  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句
```

多表查询

运行以下 SQL 语句

```
SELECT Student.Sno,Sname FROM Student,SC WHERE Student.Sno=SC.Sno AND SC.Cno='2';
```

```
SQL>>SELECT Student.Sno,Sname FROM Student,SC WHERE Student.Sno=SC.Sno AND SC.Cno='2';  
student.sno      sname  
Total row: 0
```

polarDB 运行成功。XDSQL 运行成功。

运行以下 SQL 语句

```
SELECT FIRST.Cno,SECOND.Cpno FROM Course FIRST,Course SECOND WHERE  
FIRST.Cpno=SECOND.Cno;
```

```
SELECT* FROM Student LEFT OUTER JOIN SC ON(Student.Sno=SC.Sno);
```

```
SELECT* FROM Course RIGHT OUTER JOIN SC ON(Course.Cno=SC.Cno);
```

```
SELECT* FROM Course JOIN SC ON(Course.Cno=SC.Cno);
```

```
SQL>>SELECT FIRST.Cno,SECOND.Cpno FROM Course FIRST,Course SECOND WHERE FIRST.Cpno=SECOND.Cno;  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句
```

polarDB 运行成功。XDSQL 显示”语法错误/不支持本条 SQL 语句”

嵌套查询

运行以下 SQL 语句

```
SELECT Sname FROM Student WHERE EXISTS(SELECT* FROM SC WHERE Sno=Student.Sno  
AND Cno='1');
```

```
SQL>>SELECT Sname FROM Student WHERE EXISTS(SELECT* FROM SC WHERE Sno=Student.Sno AND Cno='1');  
EXISTS语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句
```

polarDB 运行成功。XDSQL 显示”语法错误/不支持本条 SQL 语句“

视图

定义视图

运行以下 SQL 语句

```
CREATE VIEW V1 AS SELECT SC.Sno,Sname FROM Student,SC WHERE Sdept='IS' AND  
SC.Cno='1' AND Student.Sno=SC.Sno;  
  
CREATE VIEW V2 AS SELECT Sname FROM V1;
```

```
SQL>>CREATE VIEW V1 AS SELECT SC.Sno,Sname FROM Student,SC WHERE Sdept='IS' AND SC.Cno='1' AND Student.Sno=SC.Sno;  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句
```

polarDB 运行成功。XDSQL 显示”语法错误/不支持本条 SQL 语句”

删除视图

运行以下 SQL 语句

```
DROP VIEW V2;
```

```
SQL>>DROP VIEW V2;  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句
```

polarDB 运行成功。XDSQL 显示”语法错误/不支持本条 SQL 语句”

查询视图

运行以下 SQL 语句

```
SELECT Sname FROM V1 WHERE Sname LIKE '李_';
```

```
SQL>>SELECT Sname FROM V1 WHERE Sname LIKE '李_';  
LIKE语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句  
语法错误/不支持本条SQL语句
```

polarDB 运行成功。XDSQL 显示”语法错误/不支持本条 SQL 语句”

实验 3 XDSQL 数据库优化

实验环境

云主机 2C4G

操作系统 windows server 2016

实验目的

在 DBMS 原型数据库开发中，select 语句功能实现是较为复杂的，涉及到单表查询、多表查询、条件查询 和嵌套查询等功能的实现。在以往大三下学期课程编译原理的作业——实现一个简易的 DBMS 数据库系 统中，大多数学生只实现了单表查询和条件查询，多表查询只支持两张表的连接查询。为了提高学生动手 能力，加深学生对笛卡尔积和多表查询的理解，本次实验在 XDSQL 数据库源代码的基础上，提供给学 生一款只支持两张表连接查询的数据库系统 XDSQL_for_select 和其源代码，由学生将其改为支持多表查 询的数据库系统。

实验内容

XDSQL_for_select 数据库系统是对 XDSQL 数据库修改得到的，其只支持以下 SQL 语句：

```
use database [数据库名称];
```

```
show databases;
```

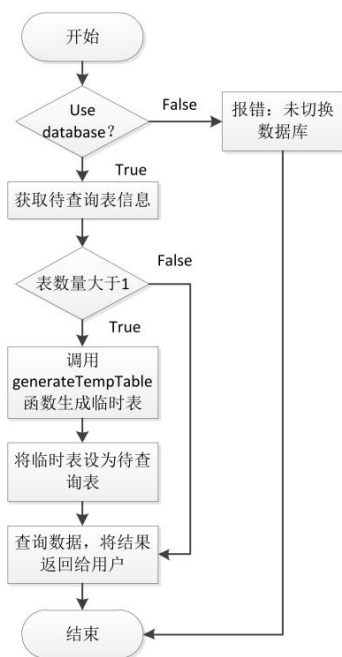
```
show tables;
```

```
select * from [表名];
```

```
select * from [表 1 名],[表 2 名];
```

该数据库不支持创建数据库、删除数据库、创建表、删除表、插入、删除、更新等操作，目前只支持单 表查询和双表查询，不支持条件查询和嵌套查询。

在 XDSQL_for_select 源代 码 中 ， 负 责 查 询 的 代 码 位 于 XDSQL_for_select/QueryProcessor.cpp 中 select 函数内，其功能流程图如下：



当执行多表查询时，程序首先对待查询的多张表进行笛卡尔积操作，将结果保存到临时表中，随后基于临时表进行查询操作，最后删除临时表。生成临时表的函数名为 `generateTempTable`，其中调用 `tablejoin` 函数用于多张表的笛卡尔积操作，并将结果存入临时表。目前 `tablejoin` 只支持两张表笛卡尔积操作。本次实验的目的便是修改 `tablejoin` 函数，使其支持多张表笛卡尔积操作。

涉及到的结构体说明：

tableNode 结构体：具体定义见 `MetadataManager.h` 文件，记录数据表的元数据信息，包括表名、表内列数、一行数据的大小（单位：字节）和指向下一个 `tableNode` 结构体的指针。

storageNode 结构体：具体定义见 `StorageEngine.h` 文件，可以将其视为指向数据表内数据行的指针。

涉及到的函数说明：

```
void tablejoin(tableNode* tablenode, char* row, int index, storageNode* sp);
```

负责对多张表进行笛卡尔积操作，并将结果通过 `sp` 写入临时表中。其传入参数如下：

tablenode：数据表节点，记录待查询数据表基本信息。

row：字节数组，作为临时数据行，负责笛卡尔积过程中数据的暂时存储。

index: 字节数组指针，负责记录下一张表数据行数据在 **row** 字节数组中存储的首地址。

sp: 临时数据表的数据行指针。

```
void open(storageNode* sp, char* tablename, int rowlength);
```

打开 **tablename** 数据表，并用 **sp** 指向该数据表内的数据行。其中 **tablename** 为数据表名称，**rowlength** 为数据表内一行数据所占字节数。

```
void rnd_init(storageNode* sp);
```

初始化 **sp** 指针，使其指向数据表第一行数据。

```
bool isEOF(storageNode* sp);
```

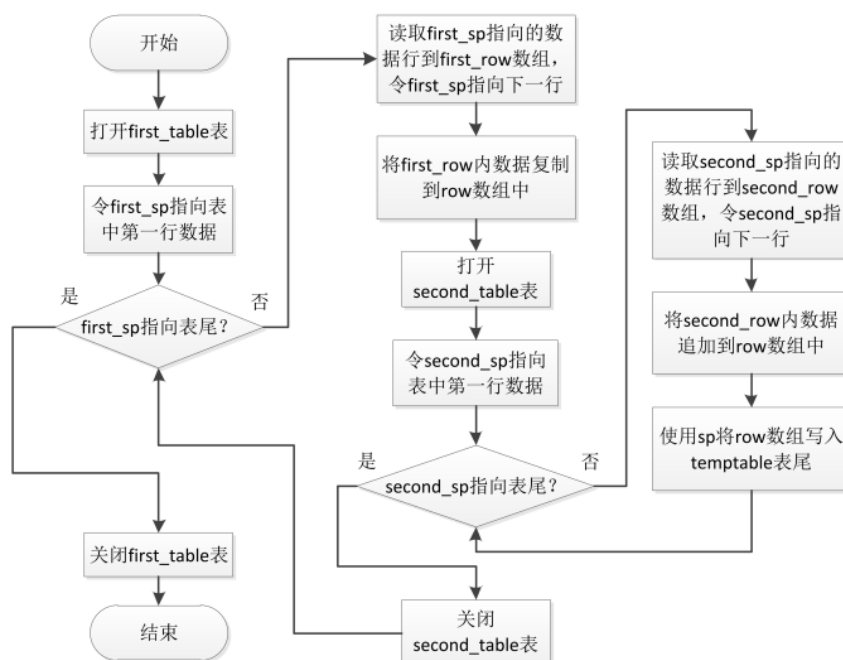
判断数据表是否读取完毕，完毕返回 **true**，否则返回 **false**。

```
char* rnd_next(storageNode* sp);
```

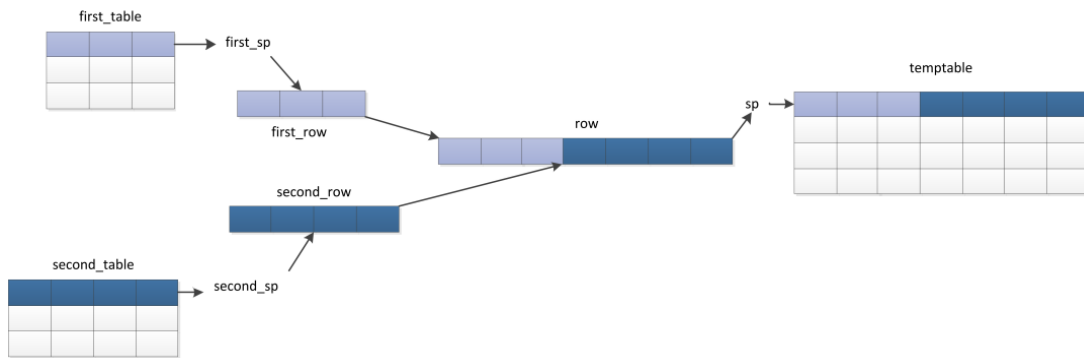
读取 **sp** 指向的数据行数据，随后令 **sp** 指向下一行数据。返回值如果为 **NULL**，说明该行数据已被删除，不对数据进行任何操作。

```
void write_row(storageNode* sp, char* row);
```

向 **sp** 指向的数据表表尾写入一行数据。**row** 字节数组记录待写入的数据行数据。**tablejoin** 函数功能如下图所示：



其中 **first_table** 和 **second_table** 为待查询的表，**temptable** 为临时表，是待查询表笛卡尔积后的结果表。下图展示了临时表第一行数据生成过程：



现需要同学们对 `tablejoin` 函数进行修改，使该函数具有多张表笛卡尔积功能，使 `newSQLdemo` 数据库支持多表查询。

实验步骤

本次实验在 windows server 2016 操作系统下进行，编译工具为 VS2019，具体步骤如下：

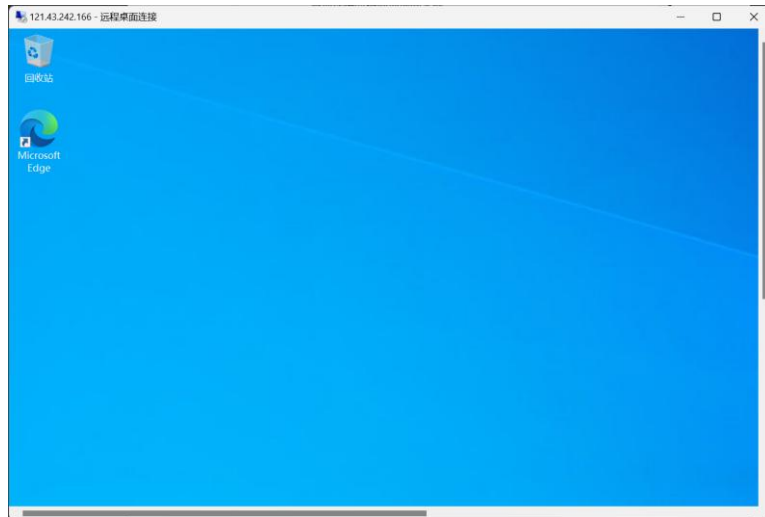
1. 申请云主机，点击创建资源，获得 ECS 公网 IP，登录名和密码



2. 打开开始菜单，搜索远程桌面连接并运行，依次填写连接信息



3. 点击连接，登录到 ECS 服务器。第一次连接可能会出现连接失败，等待一两分钟后再次连接即可登录



4. 将 Git-2.31.1-64-bit.exe 复制到云主机
5. 安装 git，安装步骤直接 Next 即可
6. 运行 cmd，输入以下指令，将 XDSQL_for_select 数据库源代码下载到云主机

```
cd C:\>mkdir code
```

```
cd code
```

```
git config --global --unset http.proxy
```

```
git clone https://github.com/flytigerman/XDSQL\_for\_select.git
```

执行结果如图所示

```
C:\Users\Administrator>cd C:\
C:\>mkdir code
C:\>cd code
C:\code>git config --global --unset http.proxy
C:\code>git clone https://github.com/flytigerman/XDSQL_for_select.git
Cloning into 'XDSQL_for_select'...
remote: Enumerating objects: 143, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 143 (delta 2), reused 1 (delta 1), pack-reused 140 (from 1)
Receiving objects: 100% (143/143), 84.01 MiB | 4.24 MiB/s, done.
Resolving deltas: 100% (51/51), done.
Updating files: 100% (53/53), done.
C:\code>_
```

7. 将 VS2019 复制到云主机，安装。安装时只安装使用 C++的桌面开发即可，时间大约 20 分钟。

8. 打开 VS2019，点击右上角的文件->打开->项目，打开 XDSQL_for_select 项目（路径为 C:\code\XDSQL_for_select），执行结果如图所示

9. 打开 QueryProcessor.cpp 文件，对 tablejoin 函数进行修改，使其支持多表连接

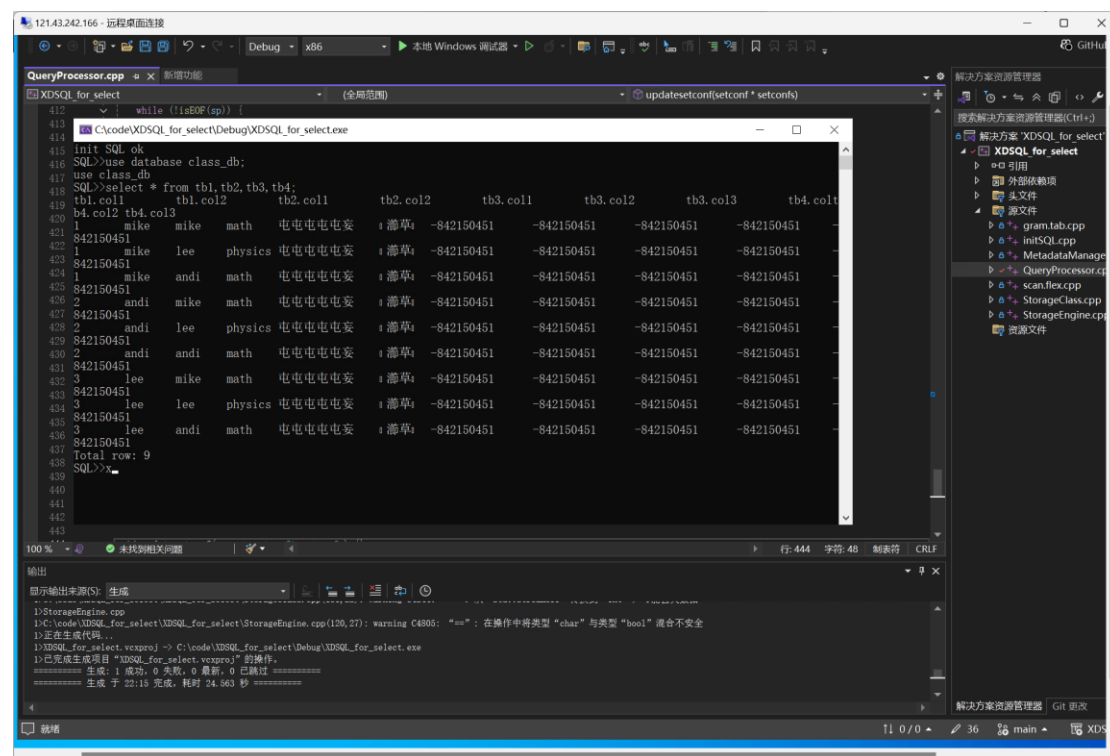
10. 修改完毕后，将 select 函数中待查询表数量大于 2 的判断条件注释或删除，如下所示

```
if (tablelist->num > 1) {  
    /*if (tablelist->num > 2) {  
        printf("error: the number of tables is not more than 2");  
        return; }*/ //删掉即可  
    tablenode = generateTempTable(tablelist);  
}
```

11. 点击运行，依次输入以下指令进行测试：

```
use database class_db;
```

```
select * from tb1,tb2,tb3,tb4;
```



公交安全管理系统设计

题目

后台数据库

设计公交安全管理系统。数据库的有关语义如下：

1. 公交公司有若干个车队，每个车队下有若干条线路；
2. 公交公司有若干辆汽车，每辆车属于一条线路；
3. 每个车队有一名队长，他只有管理工作，不开车；
4. 每条线路有若干名司机，其中有一名路队长，除开车外，还承担管理工作；每名司机只在一条线路上开车；
5. 司机开车时会产生违章，包含：闯红灯、未礼让斑马线、压线、违章停车等；
6. 队长、路队长负责将司机的违章信息输入到系统，包含：司机、车辆、车队、线路、站点、时间、违章等。

前台程序

开发一个公交安全管理系统来对数据库进行访问，可以使用 Java、Python、C 等集成开发环境。系统实现功能如下：

1. 录入司机基本信息，如工号、姓名、性别等；
2. 录入汽车基本信息，如车牌号、座数等；
3. 录入司机的违章信息；
4. 查询某个车队下的司机基本信息；
5. 查询某名司机在某个时间段的违章详细信息；
6. 查询某个车队在某个时间段的违章统计信息，如：2 次闯红灯、4 次未礼让斑马线等。

注意事项：

1. 在数据库的设计过程中需要运用规范化理论，避免出现插入异常、删除异常、数据冗余等问题；
2. 必须设定关系的完整性规则，如实体完整性（例如设置主码），参照完整性（例如设置外码和对应的主码），用户自定义完整性（例如性别只能为“男”或“女”）；

3. 使用索引来加快查询的速度；使用视图、函数、存储过程来简化系统的设计；

需求分析

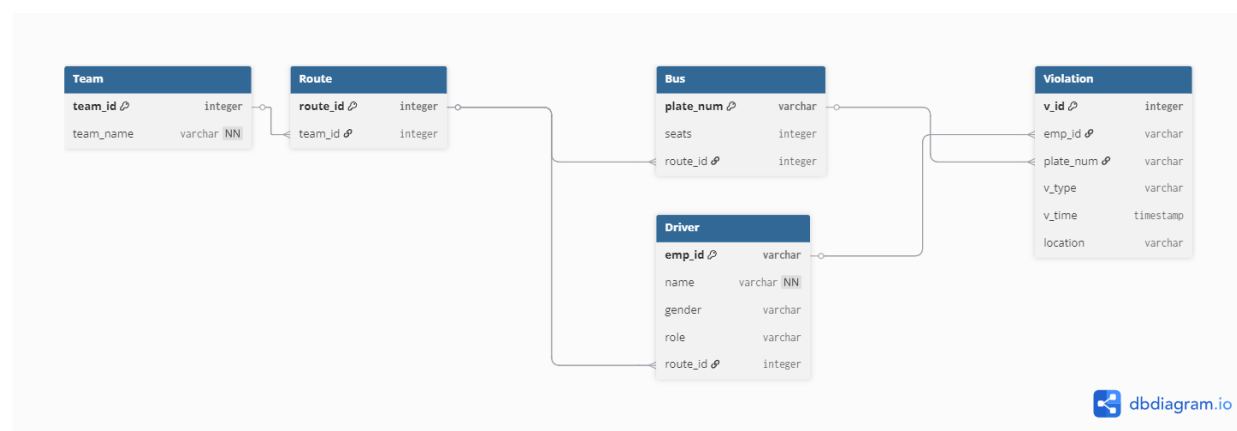
根据提供的业务规则，梳理出以下实体关系：

- 1) 车队与线路：1:N 关系（一个车队管理多条线路）。
- 2) 线路与车辆：1:N 关系（一条线路包含多辆车）。
- 3) 线路与司机：1:N 关系（一名司机只在一条线路上开车）。
- 4) 违章记录：多对多关系的体现（司机、车辆在特定时间发生的事件）。
- 5) 角色区分：队长（仅管理）、路队长（驾驶+管理）、普通司机（驾驶）。

功能需求

- 1) 数据录入：司机、车辆、违章信息。
- 2) 数据查询：按车队查司机、按司机查违章。
- 3) 统计分析：按车队、时间段统计各类违章次数。

概念结构设计 (E-R 图描述)



逻辑结构设计 (关系模型)

1) 车队表 (Team)

team_id (PK, int): 车队编号

team_name (varchar): 车队名称

2) 线路表 (Route)

route_id (PK, int): 线路号

team_id (FK): 所属车队编号

3) 车辆表 (Bus)

plate_num (PK, varchar): 车牌号

seats (int): 座位数

route_id (FK): 所属线路

4) 员工表 (Driver)

emp_id (PK, varchar): 工号

name (varchar): 姓名

gender (char): 性别 (Check: '男' or '女')

role (varchar): 职位 (队长/路队长/司机)

route_id (FK): 所属线路 (队长可为空或特定规则，此处假设关联线路)

5) 违章记录表 (Violation)

v_id (PK, integer, auto_increment): 记录编号

emp_id (FK): 司机工号

plate_num (FK): 车牌号

v_type (varchar): 违章类型 (闯红灯/压线等)

v_time (datetime): 违章时间

location (varchar): 地点

程序开发环境

操作系统: Windows

开发语言: Python 3.14

IDE: PyCharm Community

数据库: SQLite3

前端 GUI 库: Tkinter

应用程序设计中遇到的问题及解决方法

问题: 频繁打开关闭数据库影响性能。

解决：使用全局连接对象或上下文管理器 (Context Manager) 确保连接正确关闭。

问题：统计某车队违章需要关联 4 张表 (车队-线路-司机-违章)，SQL 语句复杂。

解决：使用 SQL JOIN 语句或创建视图 (VIEW) 来简化查询逻辑。

问题：用户输入时间格式不统一。

解决：前端使用简单的格式验证，数据库存储为标准字符串 (YYYY-MM-DD HH:MM:ss)。

总结

本系统完成了从需求分析、数据库设计到应用开发的完整流程。我将课堂上学习的 1NF、2NF、3NF 规范化理论应用于实际项目中，通过合理设计表结构，成功避免了数据冗余和更新异常问题。掌握了从业务需求分析到 ER 图设计的完整流程，能够将复杂的业务逻辑转化为清晰的数据模型。我积累了前后端集成开发经验，体验了完整项目开发流程，提升了问题解决能力，培养了工程实践意识。这次项目经验将成为我未来学习和职业发展的重要基石，我将继续在数据库技术领域深入探索，努力成为一名优秀的数据工程师。

附录：建立数据库和应用程序的主要代码

```
import sqlite3

import tkinter as tk

from tkinter import ttk, messagebox

from datetime import datetime

class Database:

    def __init__(self, db_file="bus_data.db"):

        self.conn = sqlite3.connect(db_file)

        self.cur = self.conn.cursor()

        self.init_db()
```

```
def init_db(self):

    """初始化数据库结构，创建表和视图"""

    self.cur.execute("PRAGMA foreign_keys = ON;")

    # 1. 车队表

    self.cur.execute("""

        CREATE TABLE IF NOT EXISTS Team (

            team_id INTEGER PRIMARY KEY,

            team_name TEXT NOT NULL

        )

    """)

    # 2. 线路表

    self.cur.execute("""

        CREATE TABLE IF NOT EXISTS Route (

            route_id INTEGER PRIMARY KEY,

            team_id INTEGER,

            FOREIGN KEY (team_id) REFERENCES Team(team_id)

        )

    """)

    # 3. 车辆表

    self.cur.execute("""

        CREATE TABLE IF NOT EXISTS Bus (

            plate_num TEXT PRIMARY KEY,

            seats INTEGER,

            route_id INTEGER,

            FOREIGN KEY (route_id) REFERENCES Route(route_id)

        )

    """)
```

```
""")
```

```
# 4. 员工表 (司机/队长)
```

```
self.cur.execute("""
```

```
    CREATE TABLE IF NOT EXISTS Driver (  
  
        emp_id TEXT PRIMARY KEY,  
  
        name TEXT NOT NULL,  
  
        gender TEXT CHECK(gender IN ('男', '女')),  
  
        role TEXT,  
  
        route_id INTEGER,  
  
        FOREIGN KEY (route_id) REFERENCES Route(route_id)  
  
    )
```

```
""")
```

```
# 5. 违章记录表
```

```
self.cur.execute("""
```

```
    CREATE TABLE IF NOT EXISTS Violation (  
  
        v_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
        emp_id TEXT,  
  
        plate_num TEXT,  
  
        v_type TEXT,  
  
        v_time TEXT,  
  
        location TEXT,  
  
        FOREIGN KEY (emp_id) REFERENCES Driver(emp_id),  
  
        FOREIGN KEY (plate_num) REFERENCES Bus(plate_num)  
  
    )
```

```
""")
```

```
# 创建索引优化查询
```

```
self.cur.execute("CREATE INDEX IF NOT EXISTS idx_v_time ON Violation(v_time);")
```

```

# 预制一些基础数据

self.preload_data()

self.conn.commit()

def preload_data(self):
    # 检查是否已有数据，无则插入

    self.cur.execute("SELECT count(*) FROM Team")

    if self.cur.fetchone()[0] == 0:

        self.cur.execute("INSERT INTO Team VALUES (1, '一车队'), (2, '二车队')")

        self.cur.execute("INSERT INTO Route VALUES (101, 1), (102, 1), (201, 2)")

        self.cur.execute("INSERT INTO Bus VALUES ('京 A88888', 45, 101), ('京 A12345', 30,
201)")

        self.cur.execute("INSERT INTO Driver VALUES ('D001', '张三', '男', '司机', 101)")

        self.cur.execute("INSERT INTO Driver VALUES ('D002', '李四', '女', '路队长', 201)")

        print("初始化测试数据完成。")

def add_driver(self, emp_id, name, gender, role, route_id):
    try:

        self.cur.execute("INSERT INTO Driver VALUES (?, ?, ?, ?, ?)",

                        (emp_id, name, gender, role, route_id))

        self.conn.commit()

        return True, "添加成功"

    except sqlite3.IntegrityError as e:

        return False, f"添加失败: {e}"

def add_bus(self, plate, seats, route_id):
    try:

        self.cur.execute("INSERT INTO Bus VALUES (?, ?, ?)", (plate, seats, route_id))

        self.conn.commit()

```

```

        return True, "添加成功"

    except sqlite3.Error as e:

        return False, str(e)

def add_violation(self, emp_id, plate, v_type, v_time, location):

    try:

        self.cur.execute("INSERT INTO Violation (emp_id, plate_num, v_type, v_time,
location) VALUES (?, ?, ?, ?, ?)",

                        (emp_id, plate, v_type, v_time, location))

        self.conn.commit()

        return True, "录入成功"

    except sqlite3.Error as e:

        return False, str(e)

def query_drivers_by_team(self, team_id):

    # 多表连接查询：车队 -> 线路 -> 司机

    sql = """

    SELECT d.emp_id, d.name, d.gender, d.role, r.route_id, t.team_name

    FROM Driver d

    JOIN Route r ON d.route_id = r.route_id

    JOIN Team t ON r.team_id = t.team_id

    WHERE t.team_id = ?

    """

    self.cur.execute(sql, (team_id,))

    return self.cur.fetchall()

def query_violation_by_driver(self, emp_id, start_date, end_date):

    sql = """

    SELECT v.v_time, v.v_type, v.location, v.plate_num

    FROM Violation v

```

```
WHERE v.emp_id = ? AND v.v_time BETWEEN ? AND ?
```

```
ORDER BY v.v_time DESC
```

```
"""
```

```
self.cur.execute(sql, (emp_id, start_date, end_date))
```

```
return self.cur.fetchall()
```

```
def stats_violation_by_team(self, team_id, start_date, end_date):
```

```
    # 统计查询
```

```
    sql = """
```

```
SELECT v.v_type, COUNT(*)
```

```
FROM Violation v
```

```
JOIN Driver d ON v.emp_id = d.emp_id
```

```
JOIN Route r ON d.route_id = r.route_id
```

```
WHERE r.team_id = ? AND v.v_time BETWEEN ? AND ?
```

```
GROUP BY v.v_type
```

```
"""
```

```
self.cur.execute(sql, (team_id, start_date, end_date))
```

```
return self.cur.fetchall()
```

```
def __del__(self):
```

```
    self.conn.close()
```

```
class App:
```

```
    def __init__(self, root):
```

```
        self.db = Database()
```

```
        self.root = root
```

```
        self.root.title("公交安全管理系统")
```

```
        self.root.geometry("800x600")
```

```
# 选项卡控件

self.notebook = ttk.Notebook(root)

self.notebook.pack(expand=True, fill='both')


# 创建各个页面

self.create_input_tab()

self.create_query_tab()

self.create_stats_tab()


def create_input_tab(self):

    frame = ttk.Frame(self.notebook)

    self.notebook.add(frame, text="数据录入")


    # --- 区域 1: 录入司机 ---

    lf1 = ttk.LabelFrame(frame, text="录入司机信息")

    lf1.pack(pady=10, padx=10, fill="x")


    ttk.Label(lf1, text="工号:").grid(row=0, column=0, padx=5)

    self.e_uid = ttk.Entry(lf1)

    self.e_uid.grid(row=0, column=1)


    ttk.Label(lf1, text="姓名:").grid(row=0, column=2, padx=5)

    self.e_name = ttk.Entry(lf1)

    self.e_name.grid(row=0, column=3)


    ttk.Label(lf1, text="性别:").grid(row=0, column=4, padx=5)

    self.e_gender = ttk.Combobox(lf1, values=["男", "女"], width=5)

    self.e_gender.grid(row=0, column=5)


    ttk.Label(lf1, text="线路 ID:").grid(row=1, column=0, padx=5)
```

```
self.e_route = ttk.Entry(lf1)

self.e_route.grid(row=1, column=1)


btn_add_driver = ttk.Button(lf1, text="添加司机", command=self.action_add_driver)

btn_add_driver.grid(row=1, column=4, columnspan=2, pady=5)


# --- 区域 2: 录入违章 ---

lf2 = ttk.LabelFrame(frame, text="录入违章信息")

lf2.pack(pady=10, padx=10, fill="x")


ttk.Label(lf2, text="司机工号:").grid(row=0, column=0)

self.v_uid = ttk.Entry(lf2)

self.v_uid.grid(row=0, column=1)


ttk.Label(lf2, text="车牌号:").grid(row=0, column=2)

self.v_plate = ttk.Entry(lf2)

self.v_plate.grid(row=0, column=3)


ttk.Label(lf2, text="违章类型:").grid(row=1, column=0)

self.v_type = ttk.Combobox(lf2, values=["闯红灯", "未礼让斑马线", "压线", "违章停车"])

self.v_type.grid(row=1, column=1)


ttk.Label(lf2, text="时间(Y-M-D):").grid(row=1, column=2)

self.v_time = ttk.Entry(lf2)

self.v_time.insert(0, datetime.now().strftime("%Y-%m-%d")) # 默认当天

self.v_time.grid(row=1, column=3)


btn_add_v = ttk.Button(lf2, text="提交违章", command=self.action_add_violation)

btn_add_v.grid(row=2, column=1, pady=10)
```



```

def create_query_tab(self):

    frame = ttk.Frame(self.notebook)

    self.notebook.add(frame, text="查询功能")

    # 查询车队司机

    lf1 = ttk.LabelFrame(frame, text="查询车队下的司机")

    lf1.pack(fill="x", padx=10, pady=5)

    ttk.Label(lf1, text="车队 ID (1, 2...):").pack(side="left", padx=5)

    self.q_team_id = ttk.Entry(lf1, width=5)

    self.q_team_id.pack(side="left")

    ttk.Button(lf1, text="查询", command=self.action_query_team).pack(side="left", padx=10)

    self.tree_driver = ttk.Treeview(frame, columns=("ID", "Name", "Gender", "Role", "Route"),
show="headings", height=5)

    self.tree_driver.heading("ID", text="工号")

    self.tree_driver.heading("Name", text="姓名")

    self.tree_driver.heading("Gender", text="性别")

    self.tree_driver.heading("Role", text="职位")

    self.tree_driver.heading("Route", text="线路")

    self.tree_driver.pack(fill="x", padx=10)

    # 查询个人违章

    lf2 = ttk.LabelFrame(frame, text="查询司机违章明细")

    lf2.pack(fill="x", padx=10, pady=5)

    ttk.Label(lf2, text="工号:").pack(side="left")

    self.q_v_uid = ttk.Entry(lf2, width=10)

    self.q_v_uid.pack(side="left")

    ttk.Button(lf2, text="查询 (本月)", command=self.action_query_driver_v).pack(side="left",
padx=10)

```

```

        self.tree_v = ttk.Treeview(frame, columns=("Time", "Type", "Loc", "Plate"),
show="headings", height=5)

        self.tree_v.heading("Time", text="时间")

        self.tree_v.heading("Type", text="违章类型")

        self.tree_v.heading("Loc", text="地点")

        self.tree_v.heading("Plate", text="车牌")

        self.tree_v.pack(fill="x", padx=10)

def create_stats_tab(self):

    frame = ttk.Frame(self.notebook)

    self.notebook.add(frame, text="统计报表")

    lf = ttk.LabelFrame(frame, text="车队违章统计")

    lf.pack(pady=10)

    ttk.Label(lf, text="车队 ID:").grid(row=0, column=0)

    self.s_team = ttk.Entry(lf, width=5)

    self.s_team.grid(row=0, column=1)

    ttk.Button(lf, text="生成统计", command=self.action_stats).grid(row=0, column=2,
padx=10)

    self.txt_stats = tk.Text(frame, height=10)

    self.txt_stats.pack(padx=10, pady=10, fill="both")

# --- 事件处理函数 ---

def action_add_driver(self):

    # 简单获取数据，实际需更严格校验

```

```

        success, msg = self.db.add_driver(self.e_uid.get(), self.e_name.get(),

                                           self.e_gender.get(), "司机", self.e_route.get())

        messagebox.showinfo("结果", msg)

    def action_add_violation(self):

        success, msg = self.db.add_violation(self.v_uid.get(), self.v_plate.get(),

                                           self.v_type.get(), self.v_time.get(), "市区
道路")

        messagebox.showinfo("结果", msg)

    def action_query_team(self):

        # 清空表格

        for row in self.tree_driver.get_children():

            self.tree_driver.delete(row)

        tid = self.q_team_id.get()

        if not tid: return

        res = self.db.query_drivers_by_team(tid)

        for r in res:

            self.tree_driver.insert("", "end", values=r)

    def action_query_driver_v(self):

        for row in self.tree_v.get_children():

            self.tree_v.delete(row)

        uid = self.q_v_uid.get()

        # 简化处理，默认查询 2020 年到 2030 年

        res = self.db.query_violation_by_driver(uid, "2020-01-01", "2030-12-31")

        for r in res:

            self.tree_v.insert("", "end", values=r)

```

```

def action_stats(self):

    tid = self.s_team.get()

    if not tid: return

    res = self.db.stats_violation_by_team(tid, "2020-01-01", "2030-12-31")

    self.txt_stats.delete(1.0, "end")

    self.txt_stats.insert("end", f"车队 {tid} 违章统计 (2020-2030):\n")

    self.txt_stats.insert("end", "-"*30 + "\n")

    if not res:

        self.txt_stats.insert("end", "无记录")

    for r in res:

        self.txt_stats.insert("end", f"类型: {r[0]} \t 次数: {r[1]}\n")

if __name__ == "__main__":

    root = tk.Tk()

    app = App(root)

    root.mainloop()

```

